

Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Punch Card

Processamento de Linguagens

Ano Letivo 2025/2026

Grupo 09

A106919 Eduardo Freitas Fernandes

A106842 Gonalo Rodrigues Ribeiro

A106888 Jos  M rio Raimundo Lima

PL

Conteúdo

1	Introdução	3
2	Arquitetura do Sistema	3
2.1	Análise Léxica (<i>lexer</i> /)	3
2.2	Análise Sintática (<i>parser</i> /)	4
2.3	Análise Semântica (<i>semantic</i> /)	4
2.4	Geração de Código (<i>codegen</i> /)	4
3	<i>Context Free Grammar</i>	5
3.1	Programa e Unidades de Programa	5
3.2	Corpo do Programa e Declarações	5
3.3	Instruções e <i>Labels</i>	6
3.4	Estruturas Condicionais	6
3.5	Ciclos DO	7
3.6	Entrada, Saída e Listas de Argumentos	7
3.7	Expressões	8
4	Decisões de Implementação	9
4.1	Uso de AST e padrão <i>Visitor</i>	9
4.2	Estruturas de controlo	9
4.3	Funções e Subrotinas	9
4.4	Resolução da ambiguidade array/função	10
4.5	Funções built-in	10
4.6	Tipos numéricos e coerção	10
5	Manual de Utilização	10
5.1	Requisitos	10
5.2	Compilar um programa Fortran	10
5.3	Executar os testes	11
5.4	Estrutura do repositório	11
6	Conclusão	11

1 Introdução

Este relatório descreve o desenvolvimento do compilador PunchCard, realizado no âmbito da unidade curricular de Processamento de Linguagens do ano letivo 2025/2026. O objetivo do projeto foi implementar um compilador completo para a linguagem Fortran 77 (standard ANSI X3.9-1978), capaz de traduzir código Fortran para instruções da máquina virtual EWVM disponibilizada. O compilador foi desenvolvido em Python, utilizando a biblioteca `ply` para a construção do analisador léxico e sintático. A implementação cobre todas as etapas obrigatórias, análise léxica, sintática, semântica e geração de código, bem como a implementação de funções e subrotinas.

2 Arquitetura do Sistema

O compilador está organizado em quatro módulos principais, cada um responsável por uma fase distinta da compilação. O fluxo de execução é sequencial: a saída de cada fase é a entrada da seguinte.

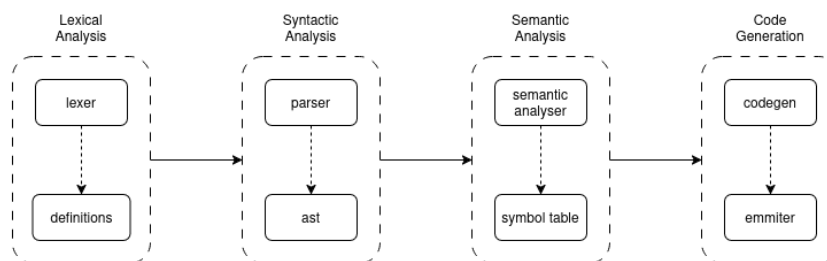


Figura 1: Arquitetura do compilador PunchCard.

2.1 Análise Léxica (lexer/)

O analisador léxico converte o código-fonte Fortran numa sequência de *tokens*. Foi implementado com `ply.lex` no módulo `lexer/lexer.py`, com as definições de tokens em `lexer/definitions.py`. Os tokens reconhecidos incluem:

- **Keywords:** PROGRAM, INTEGER, REAL, LOGICAL, CHARACTER, DOUBLE PRECISION, IF, THEN, ELSE, ENDIF, DO, CONTINUE, GOTO, PRINT, READ, FUNCTION, SUBROUTINE, RETURN, CALL, END.
- **Valores Literais:** inteiros (LIT_INT), reais (LIT_FLOAT), dupla precisão (LIT_DOUBLE), strings (LIT_STRING) e booleanos (LIT_BOOLEAN, i.e., `.TRUE.` e `.FALSE.`).
- **Operadores relacionais e lógicos:** `.EQ.`, `.NE.`, `.LT.`, `.LE.`, `.GT.`, `.GE.`, `.AND.`, `.OR.`, `.NOT.`
- **Identificadores** e caracteres literais simples: `( ) , = + - * /`.

O lexer opera em formato livre (*free-form*), ignorando espaços e tabulações. Os identificadores e palavras-chave são tratados de forma *case-insensitive*, tudo é convertido para maiúsculas internamente.

2.2 Análise Sintática (parser/)

O analisador sintático valida a estrutura gramatical do programa e constrói a Árvore Sintática Abstrata (AST). Foi implementado com `ply.yacc` no módulo `parser/parser.py`, com os nós da AST definidos em `parser/ast.py`. A gramática suporta:

- Programas principais (PROGRAM . . . END) e subprogramas (FUNCTION, SUBROUTINE).
- Declarações de variáveis escalares e arrays.
- Atribuições, expressões aritméticas, relacionais e lógicas.
- Estruturas de controlo: IF-THEN-ELSE-ENDIF, IF inline, ciclos DO com e sem passo, GOTO.
- Operações de I/O: PRINT e READ.
- Chamadas de subrotinas (CALL) e funções como expressão.

A precedência de operadores é declarada explicitamente no parser, seguindo a hierarquia do Fortran 77: potência tem maior precedência, seguida de multiplicação/divisão, adição/subtração, operadores relacionais e por fim lógicos.

2.3 Análise Semântica (semantic/)

O analisador semântico percorre a AST e verifica a coerência do programa. É implementado com o padrão *Visitor*, no módulo `semantic/semantic_analyser.py`. A tabela de símbolos está em `semantic/symbol_table.py`. As verificações realizadas incluem:

- **Declaração de variáveis:** uso de variáveis não declaradas é reportado como erro.
- **Tipos:** verificação de compatibilidade de tipos em atribuições e operações. Coerções implícitas entre tipos numéricos (INTEGER, REAL, DOUBLE PRECISION) são permitidas, com aviso.
- **Arrays:** verificação do número de índices na declaração e no acesso.
- **Labels DO/CONTINUE:** o label do DO tem de coincidir com o do CONTINUE correspondente.
- **Subprogramas:** verificação do número de argumentos em chamadas de funções e subrotinas.
- **Resolução de ambiguidade:** a sintaxe `nome(args)` é ambígua entre acesso a array e as chamadas de funções. O analisador semântico resolve esta ambiguidade consultando a tabela de símbolos e reclassifica o nó da AST em conformidade.

2.4 Geração de Código (codegen/)

O gerador de código percorre a AST e emite instruções EWVM. É também implementado com o padrão *Visitor*, no módulo `codegen/codegen.py`. O módulo `codegen/emitter.py` acumula as instruções e produz o código final como texto. As principais decisões de geração de código são descritas na secção seguinte.

3 Context Free Grammar

3.1 Programa e Unidades de Programa

```
program : program_unit_list

program_unit_list : program_unit
                  | program_unit_list program_unit

program_unit : main_program
              | function_subprogram
              | subroutine_subprogram

main_program : PROGRAM IDENTIFIER body END

function_subprog : type_spec FUNCTION IDENTIFIER '(' param_list ')' body END

subroutine_subprog : SUBROUTINE IDENTIFIER '(' param_list ')' body END
```

A lista de unidades de programa usa recursividade à esquerda para otimizar o funcionamento do *parser* LALR(1) do PLY, evitando crescimento desnecessário da pilha. O compilador interpreta um programa como um conjunto de unidades independentes, cada uma delimitada pelo seu cabeçalho e pelo *token* de encerramento correspondente.

3.2 Corpo do Programa e Declarações

```
body : declaration_section statement_section
declaration_section : declaration_section declaration
                  |

declaration : type_spec var_list
type_spec : DT_INTEGER
          | DT_REAL
          | DT_DOUBLE_PRECISION
          | DT_LOGICAL
          | DT_CHARACTER

var_list : var_decl
         | var_list ',' var_decl

var_decl : IDENTIFIER
         | IDENTIFIER '(' dim_list ')

dim_list : expr
         | dim_list ',' expr
```

O corpo do programa impõe que todas as variáveis sejam declaradas antes das instruções. A produção vazia permite subprogramas sem declarações explícitas. A regra de declaração unifica variáveis escalares e *arrays*, validando dimensões como expressões.

3.3 Instruções e *Labels*

```
statement_section : statement_section statement
                  |

statement : LIT_INT statement_body
          | statement_body

statement_body : assignment_stmt
              | goto_stmt
              | if_stmt
              | do_stmt
              | continue_stmt
              | print_stmt
              | read_stmt
              | stop_stmt
              | return_stmt
              | call_stmt

assignment_stmt : lvalue '=' expr

lvalue : IDENTIFIER
       | IDENTIFIER '(' arg_list ')'
```

Qualquer instrução pode ser precedida por um literal inteiro, representando *labels* usadas em desvios e ciclos. A ambiguidade entre chamadas de função e acessos a arrays é mantida na gramática e resolvida apenas na análise semântica através da tabela de símbolos.

3.4 Estruturas Condicionais

```
if_stmt : IF '(' expr ')' THEN statement_section ENDIF
        | IF '(' expr ')' THEN statement_section ELSE statement_section ENDIF
        | IF '(' expr ')' statement_body
```

A gramática inclui a ambiguidade clássica do *dangling else*. O PLY resolve automaticamente o conflito através de shift, associando o ELSE ao IF mais interno, garantindo o comportamento esperado.

3.5 Ciclos DO

```
do_stmt : DO LIT_INT IDENTIFIER '=' expr ',' expr do_body
        | DO LIT_INT IDENTIFIER '=' expr ',' expr ',' expr do_body

do_body : do_inner_stmts LIT_INT CONTINUE

do_inner_stmts : do_inner_stmts do_inner_statement
               |

do_inner_statement : assignment_stmt | goto_stmt | if_stmt | do_stmt
                  | print_stmt | read_stmt | stop_stmt | return_stmt |
                    call_stmt
                  | LIT_INT assignment_stmt | LIT_INT goto_stmt | LIT_INT
                    if_stmt
                  | LIT_INT do_stmt | LIT_INT print_stmt | LIT_INT read_stmt
                  | LIT_INT stop_stmt | LIT_INT return_stmt | LIT_INT
                    call_stmt
```

O corpo do DO é isolado numa regra própria para garantir que o ciclo termina na *label* indicada e que a instrução CONTINUE finaliza corretamente o bloco. Esta separação evita reduções prematuras e mantém o *parser* estável.

3.6 Entrada, Saída e Listas de Argumentos

```
continue_stmt : CONTINUE
print_stmt : PRINT '*' ',' output_list
           | PRINT LIT_STRING ',' output_list
           | PRINT '*'

read_stmt : READ '*' ',' input_list
          | READ '*'

stop_stmt : STOP
return_stmt : RETURN
call_stmt : CALL IDENTIFIER '(' arg_list ')

param_list : param_list_mult |
param_list_mult : IDENTIFIER | param_list_mult ',' IDENTIFIER

arg_list : arg_list_mult |
arg_list_mult : expr | arg_list_mult ',' expr

output_list : expr | output_list ',' expr
input_list : lvalue | input_list ',' lvalue
```

A gramática distingue listas de parâmetros formais das listas de argumentos reais, ambas permitindo ausência de elementos. As instruções de leitura e escrita suportam o formato simples (*).

3.7 Expressões

```
expr : logical_expr

logical_expr : logical_expr LOP_OR logical_and
              | logical_and

logical_and : logical_and LOP_AND logical_not
             | logical_not

logical_not : LOP_NOT logical_not
            | comparison

comparison : comparison rel_op arith_expr
           | arith_expr

rel_op : LOP_EQ | LOP_NE | LOP_LE | LOP_LT | LOP_GE | LOP_GT

arith_expr : arith_expr arith_op term
           | term

arith_op : '+' | '-'

term : term term_op factor
     | factor

term_op : '*' | '/'

factor : unary OP_POWER factor
       | unary

unary : '-' unary
       | '+' unary
       | primary

primary : '(' expr ')'
        | LIT_INT | LIT_FLOAT | LIT_DOUBLE
        | LIT_STRING | LIT_BOOLEAN
        | IDENTIFIER
        | IDENTIFIER '(' arg_list ')'
```

A hierarquia das expressões segue a precedência natural dos operadores, da lógica até aos elementos primários. A recursividade à esquerda garante associatividade correta para operações aditivas e multiplicativas, enquanto a potenciação usa recursividade à direita para refletir a sua associatividade natural. Literais, variáveis e expressões entre parênteses formam a base da precedência.

4 Decisões de Implementação

4.1 Uso de AST e padrão *Visitor*

Optámos por construir uma AST explícita em vez de gerar código diretamente durante o parsing. Isto permitiu separar claramente as fases de análise semântica e geração de código, tornando o compilador mais fácil de testar e estender.

Tanto o analisador semântico como o gerador de código implementam o padrão *Visitor*: cada nó da AST tem um método `accept(visitor)` que invoca o método correspondente no visitante (e.g., `visit_IfStmt`, `visit_DoStmt`). Isto evita misturar lógica de múltiplas fases no mesmo código.

4.2 Estruturas de controlo

Ciclo DO. O ciclo DO em Fortran 77 tem a forma `DO label var = start, stop [, step]`, terminando num `label CONTINUE`. Na geração de código, o ciclo é traduzido para:

1. Inicialização da variável de controlo.
2. Label de início do ciclo.
3. Teste de condição (`var <= stop`).
4. salto condicional para o fim se falso.
5. Corpo do ciclo.
6. Incremento da variável de controlo (passo 1 por omissão).
7. Salto incondicional para o início.
8. Label de fim do ciclo.

IF. O IF bloco (IF-THEN-ELSE-ENDIF) e o IF inline são tratados de forma uniforme. Na geração de código, geram-se dois labels internos (*ifelse* e *ifend*) para controlar o fluxo com instruções JZ e JUMP.

GOTO. O GOTO é traduzido diretamente para `JUMP LN`, onde N é o label numérico do Fortran. Os statements com label emitem o label correspondente no início.

4.3 Funções e Subrotinas

O suporte a subprogramas foi implementado como valorização. Funções e subrotinas são emitidas após o programa principal, cada uma com o seu label.

Funções. Em Fortran 77, o valor de retorno de uma função é atribuído a uma variável com o mesmo nome da função (e.g., `CONVRT = VAL`). Essa variável é declarada implicitamente no início do frame da função. Na chamada, é reservado espaço no stack para o valor de retorno antes de adicionar os argumentos.

Subrotinas. As subrotinas não devolvem valor. São invocadas com `CALL`, e os argumentos são limpos do stack após o retorno.

Passagem de argumentos. Os argumentos são passados por valor, são colocados na *stack* antes da chamada. Dentro do subprograma, são acedidos como locais negativos relativamente ao *frame pointer* (PUSHL com índice negativo).

4.4 Resolução da ambiguidade array/função

A sintaxe `nome(args)` é sintacticamente idêntica para acessos a *arrays* e para chamadas de funções. O *parser* produz sempre um nó `ArrayAccess`. O analisador semântico, ao visitar esse nó, consulta a tabela de símbolos: se `nome` for uma função, reclassifica o nó para `FunctionCall` e ajusta os atributos. O gerador de código trata os dois casos de forma distinta.

4.5 Funções built-in

As funções intrínsecas `MOD`, `SIN` e `COS` são tratadas diretamente no gerador de código, mapeando para as instruções EWVM correspondentes (`MOD`, `FSIN`, `FCOS`), sem necessitar de declaração explícita.

4.6 Tipos numéricos e coerção

A hierarquia de tipos numéricos seguida é: `INTEGER < REAL < DOUBLE PRECISION`. Em operações mistas, o resultado é promovido para o tipo mais alto. Atribuições com coerção implícita são permitidas mas geram um aviso semântico.

5 Manual de Utilização

5.1 Requisitos

- Python 3.10 ou superior.
- Dependências listadas em `requirements.txt` (instalar com `pip install -r requirements.txt`).

5.2 Compilar um programa Fortran

Para compilar um ficheiro `.f` e obter o código EWVM, deverá dirigir-se à pasta `src` e executar:

```
python -m punchcard.main <ficheiro.f>
```

Exemplo:

```
python -m punchcard.main ../test/basic/01_hello.f
```

O código EWVM é impresso para o *stdout*. Em caso de erros léxicos, sintáticos ou semânticos, são impressas mensagens de erro com número de linha e coluna, e o compilador termina sem gerar código. Também é possível passar código pelo *stdin*:

```
cat programa.f | python -m punchcard.main
```

5.3 Executar os testes

O script `run_tests.sh` corre todos os testes e mostra o resultado:

```
bash run_tests.sh all
```

Podem também ser executados grupos específicos:

```
bash run_tests.sh basic    # exemplos do enunciado
bash run_tests.sh custom   # testes de funcionalidades individuais
```

5.4 Estrutura do repositório

- `src/punchcard/` - código-fonte do compilador.
- `test/basic/` - os 5 exemplos do enunciado.
- `test/custom/` - testes de funcionalidades específicas.
- `docs/` - relatório técnico.
- `assets/` - enunciado do trabalho prático.
- `CFG.md` - *Context Free Grammar*.
- `EWVM.md` - linguagem baseada em pilhas da máquina virtual.
- `run_tests.sh` - *script* para execução dos testes.

6 Conclusão

O compilador PunchCard implementa com sucesso todas as etapas obrigatórias do enunciado, bem como a valorização de subprogramas. Todos os exemplos do enunciado são compilados e produzem o resultado correto, e os 11 testes adicionais passam sem falhas. As principais dificuldades encontradas foram:

- A **ambiguidade entre acesso a array e chamada de função**: resolvida na fase semântica com reclassificação dos nós da AST.
- O **mecanismo de retorno de funções** em Fortran 77, onde o valor de retorno é atribuído a uma variável com o mesmo nome da função, exigiu tratamento especial tanto na tabela de símbolos como na geração de código.
- A **passagem de argumentos** a subprogramas, incluindo a indexação correta relativa ao *frame pointer*.

No geral, o projeto permitiu consolidar os conceitos de processamento de linguagens, desde a especificação léxica e gramatical até à geração de código para uma máquina virtual real.